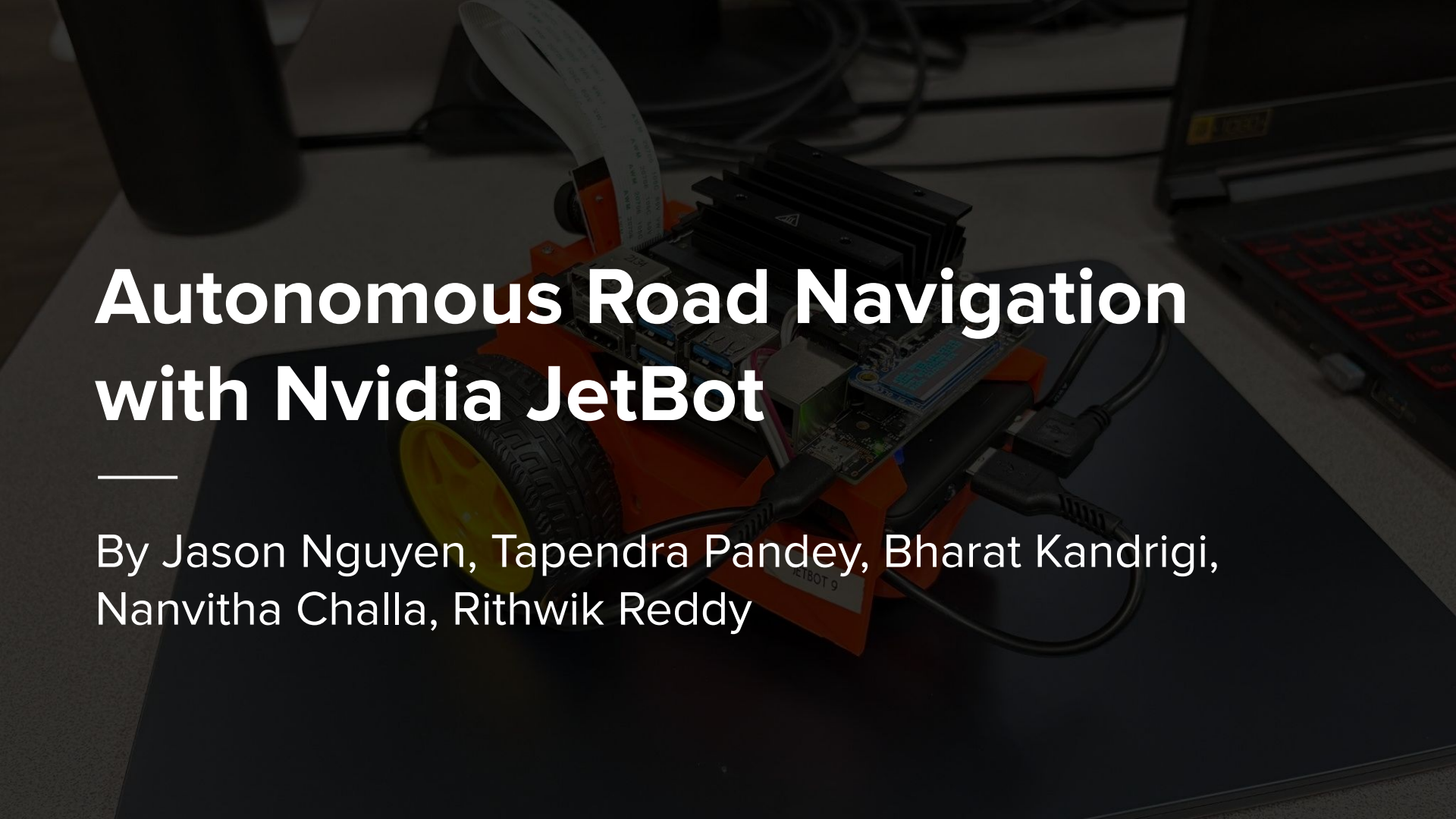


Autonomous Road Navigation with Nvidia JetBot

By Jason Nguyen, Tapendra Pandey, Bharat Kandrigi, Nanvitha Challa, Rithwik Reddy

Overview

This project demonstrates how to implement autonomous path following on **JetBot** (a small- scale robotic system powered by NVIDIA Jetson Nano)

Problem Statement

Traditional path following relies on explicit line detection or multiple sensors, making the system complex and less adaptable.

Problem Statement

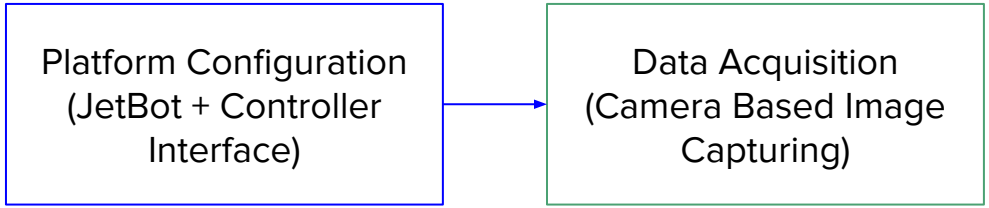
Traditional path following relies on explicit line detection or multiple sensors, making the system complex and less adaptable.

Our approach replaces this with end-to-end learning, where a neural network maps camera input directly to steering commands, simplifying the pipeline and enabling easy retraining for different path types.

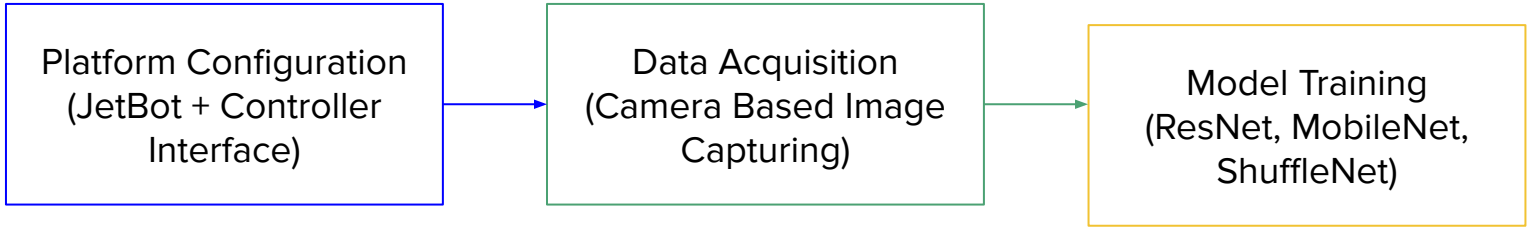
Methodology

Platform Configuration (JetBot + Controller Interface)

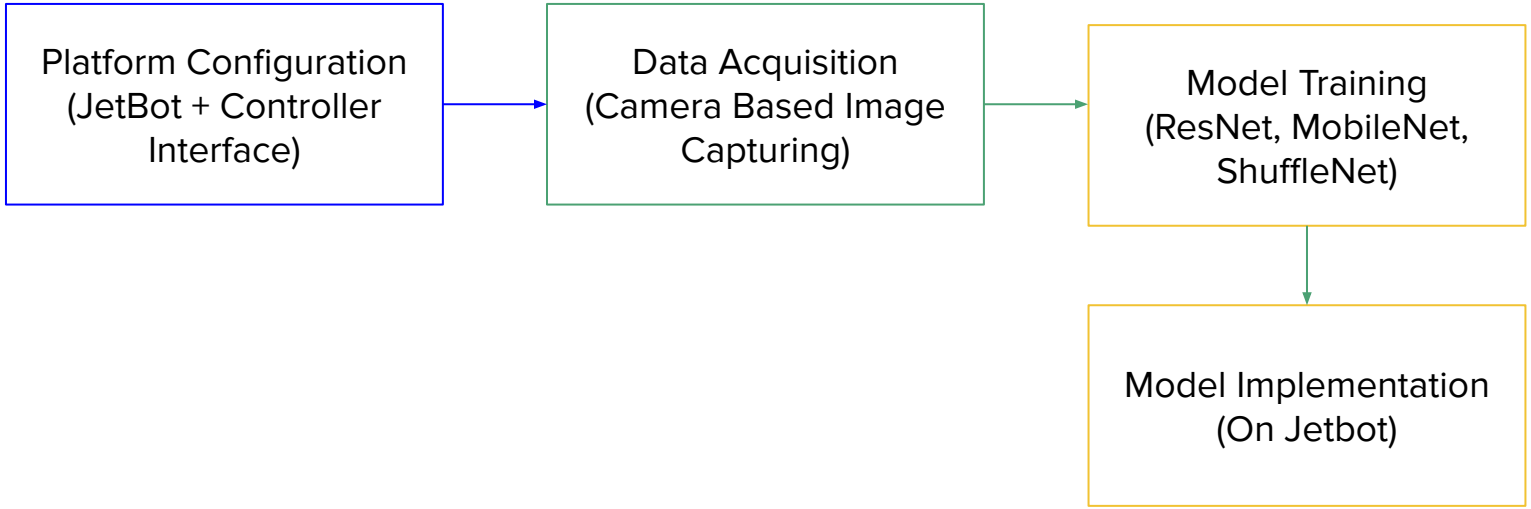
Methodology



Methodology



Methodology



Models

Research involved finding what different models to use. We came across several, and used the following:

- Resnet18
- Shufflenet
- Mobilenet

Models (MobileNet)

Model Configuration:

- The model used is torchvision.models.mobilenet_v2(pretrained=True).
- MobileNetV2 is a lightweight architecture designed for efficient inference on mobile and edge devices like Jetson Nano.

The original classifier (used for ImageNet's 1000 classes) is replaced with:

```
model.classifier[1] = nn.Linear(model.last_channel, 2)
```

- This modification adapts the network to predict 2 continuous outputs (X and Y steering coordinates).

Models (MobileNet)

Dataset and Preprocessing:

- A custom dataset class XYDataset is defined. It:
    - Loads .jpg images and parses X and Y labels from filenames.
    - Applies data augmentation: color jittering and random horizontal flips.
    - Resizes all images to 224×224, then normalizes using ImageNet’s mean and std.
    - Flipped images have X-coordinate negated to maintain semantic correctness.

Models (MobileNet)

Training Loop Details:

- For each epoch:
    - The model is set to train() mode.
    - All batches are passed through the model.
    - MSE loss is computed and back propagated.
- After training, it enters eval() mode to compute validation loss.
- Best model (lowest validation loss) is saved to disk.

Models (MobileNet)

Why MobileNetV2?

- We selected this model for deployment due to its efficient performance, low inference latency, and consistent steering accuracy across trials.

- Despite having fewer parameters (~3.4M), it performs well in our road-following task.

Models (ResNet18)

Network Depth & Design:

- ResNet18 has 18 layers with residual blocks and skip connections.
- Known for its stability and high capacity to learn visual features.

Models (ResNet18)

Training Setup:

- **Epochs**: 70 — significantly longer than MobileNet/ShuffleNet
- **Batch size**: 8
- **Optimizer**: Adam
- **Loss**: MSE
- Same dataset and preprocessing pipeline as MobileNet

Models (ResNet18)

Why ResNet18?

- Provides strong feature extraction, particularly helpful in complex scenarios.
- However, training time and inference are heavier on Jetson Nano.
- The model contains ~11.7 million parameters, making it computationally expensive.

Models (ResNet18)

Deployment Tradeoffs:

- While training accuracy might be high, real-time performance is compromised.
- Best used for experimentation, not for edge-device deployment.

Models (ShuffleNet)

Network Design:

- ShuffleNet uses channel shuffling and pointwise group convolutions for reducing computation.
- Parameters: ~2.3 million — lighter than both MobileNet and ResNet.

Models (ShuffleNet)

Training Setup:

- **Epochs**: 18
- **Batch Size**: 8
- **Loss Function**: MSE
- **Optimizer**: Adam

Models (ShuffleNet)

Why ShuffleNet?

- Performs faster than ResNet18 and is comparable to MobileNet.
- However, the results in your testing phase showed slightly lower stability than MobileNet.
- Great alternative when model size and inference speed are top priority, but needs careful tuning.

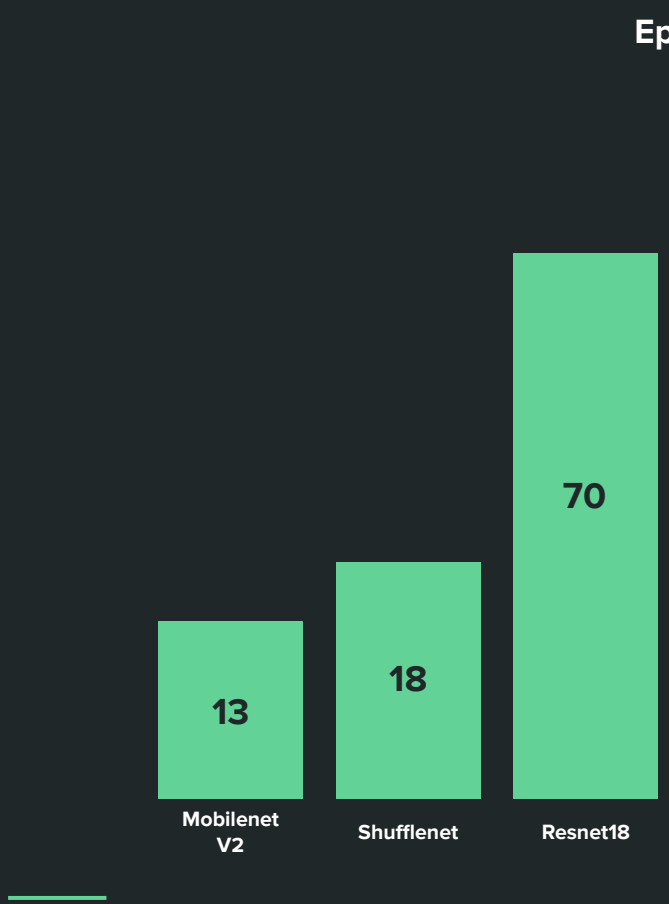
Comparison

Model	Parameters	Epochs	Inference Speed	Accuracy	Jetson Suitability
ResNet18	~11.7M	70	Slow	High	(Too heavy)
ShuffleNet	~2.3M	18	Fast	Moderate	Good
MobileNetV2	~3.4M	13	Fastest	Stable	Best choice

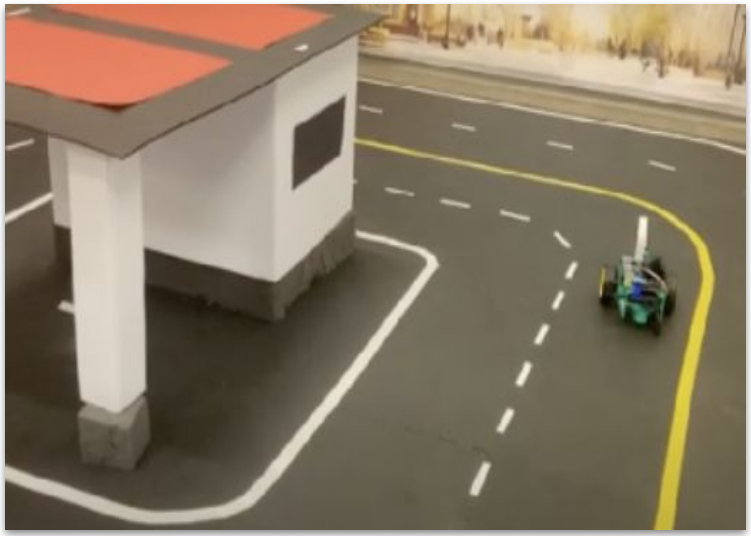
Training

Certain models require a specific number of epochs to avoid issues such as overfitting during training.

We found that smaller models needed less training epochs, as shown in the diagram



Testing Methodology



Evaluation Steps:

- We had the JetBot complete the route three times in succession.
- If the JetBot deviated from the route at any point, it was considered a failure and the test was restarted.
- A shorter total time to complete the three loops indicated better performance.

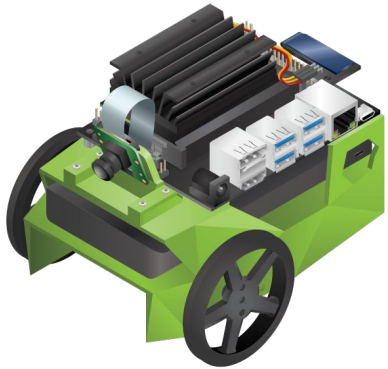
Findings

Overall, we found that ResNet18 was the most robust. The highest speed gain at which the JetBot successfully completed three loops was 0.55. MobileNetV2 ranked second, with a maximum speed gain of 0.50, while ShuffleNet came last, with a top speed gain of 0.32.

A comparison of the 3 models is shown below:

TABLE I: Comparison of Neural Network Models for Path Following

Model	Train MSE	Test MSE	Model Size (MB)	Training Epochs
ResNet18	0.002983	0.014444	44.8	70
MobileNetV2	0.042191	0.060652	9.2	13
ShuffleNet	0.005306	0.011637	5.2	18



Conclusion

Resnet performed the best overall, while mobilenet came in second and shufflenet came in third.

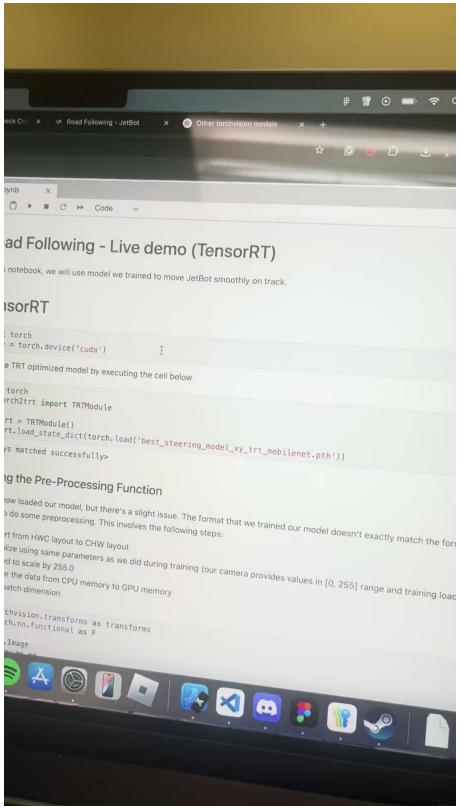
Future Works

- Model Optimization
- Custom Training
- Multi-model Integration
- Real-time Navigation
- Cloud Offloading

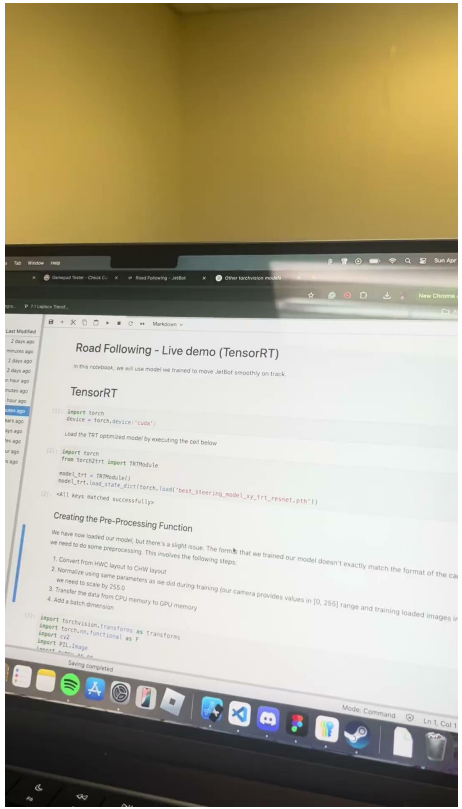
References

- L. Li, H. P. H. Shum, and T. P. Breckon, “Less is more: Reducing task and model complexity for 3d point cloud semantic segmentation,” 2023. [Online]. Available: https://arxiv.org/abs/2303.11203
- NVIDIA, “NVIDIA TensorRT,” https://developer.nvidia.com/tensorrt, 2025, accessed: 2025-04-30. [Online]. Available: https://developer.nvidia.com/tensorrt

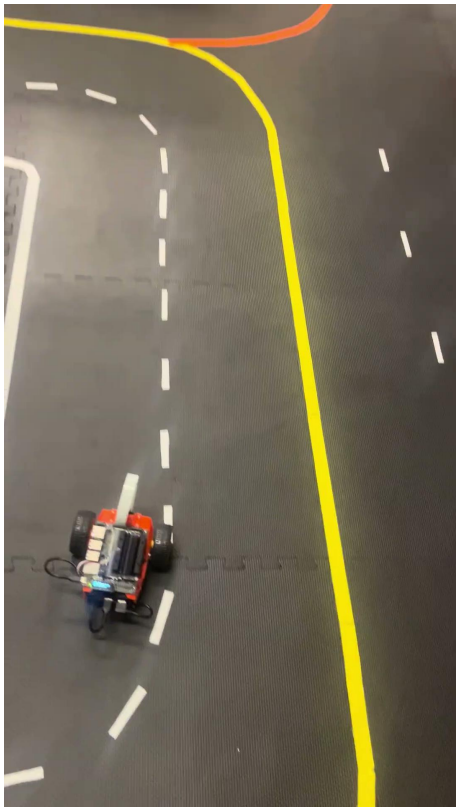
Demo



MobileNet- *30 sec*



ResNet - *23sec*



ShuffleNet - *40sec*